

```
In [68]: import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
```

```
In [69]: # Reading the first dataset containing movies meta data as movie_metadata.csv
df1=pd.read_csv(r'D:/DataMining_coursework/movie_metadata.csv')
#Reading the second dataset containing the ratings of movies on netflix
df2=pd.read_csv(r'D:/DataMining_coursework/Netflix_Dataset_Movie.csv')
```

```
In [70]: # info about dataset before cleaning and stuff
# Dataset 1
print("INFO OF DATASET ONE")
df1.info()
print("-" * 40)

# Dataset 2
print("INFO OF DATASET TWO")
df2.info()
```

INFO OF DATASET ONE

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 5043 entries, 0 to 5042

Data columns (total 28 columns):

#	Column	Non-Null Count	Dtype
0	color	5024 non-null	object
1	director_name	4939 non-null	object
2	num_critic_for_reviews	4993 non-null	float64
3	duration	5028 non-null	float64
4	director_facebook_likes	4939 non-null	float64
5	actor_3_facebook_likes	5020 non-null	float64
6	actor_2_name	5030 non-null	object
7	actor_1_facebook_likes	5036 non-null	float64
8	gross	4159 non-null	float64
9	genres	5043 non-null	object
10	actor_1_name	5036 non-null	object
11	movie_title	5043 non-null	object
12	num_voted_users	5043 non-null	int64
13	cast_total_facebook_likes	5043 non-null	int64
14	actor_3_name	5020 non-null	object
15	facenumber_in_poster	5030 non-null	float64
16	plot_keywords	4890 non-null	object
17	movie_imdb_link	5043 non-null	object
18	num_user_for_reviews	5022 non-null	float64
19	language	5029 non-null	object
20	country	5038 non-null	object
21	content_rating	4740 non-null	object
22	budget	4551 non-null	float64
23	title_year	4935 non-null	float64
24	actor_2_facebook_likes	5030 non-null	float64
25	imdb_score	5043 non-null	float64
26	aspect_ratio	4714 non-null	float64
27	movie_facebook_likes	5043 non-null	int64

dtypes: float64(13), int64(3), object(12)

memory usage: 1.1+ MB

INFO OF DATASET TWO

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 17770 entries, 0 to 17769

Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
0	Movie_ID	17770 non-null	int64
1	Year	17770 non-null	int64
2	Name	17770 non-null	object

dtypes: int64(2), object(1)

memory usage: 416.6+ KB

```
In [71]: #displaying the firstdataset
df1.head()
```

Out[71]:

	color	director_name	num_critic_for_reviews	duration	director_facebook_likes	actor
0	Color	James Cameron	723.0	178.0	0.0	
1	Color	Gore Verbinski	302.0	169.0	563.0	
2	Color	Sam Mendes	602.0	148.0	0.0	
3	Color	Christopher Nolan	813.0	164.0	22000.0	
4	NaN	Doug Walker	NaN	NaN	131.0	

5 rows × 28 columns



In [72]: `#displaying the second dataset`
`df2.head()`

Out[72]:

	Movie_ID	Year	Name
0	1	2003	Dinosaur Planet
1	2	2004	Isle of Man TT 2004 Review
2	3	1997	Character
3	4	1994	Paula Abdul's Get Up & Dance
4	5	2004	The Rise and Fall of ECW

Identify and handle missing values and duplicate records.

```
In [73]: # handling the null values
print(df1.isnull().sum())
print(df2.isnull().sum())

# dropping null values
'''
Instead of imputing the dataset with mean and modes we just drop the rows
that have NA values as imputing would result in the loss of uniqueness of each m
in my opinion is harmful for downstream analysis tasks
'''
df1=df1.dropna(axis=0)

#dropping duplicates in place if any present
df1.drop_duplicates(inplace=True)
df2.drop_duplicates(inplace=True)
```

```

color                19
director_name        104
num_critic_for_reviews 50
duration             15
director_facebook_likes 104
actor_3_facebook_likes 23
actor_2_name         13
actor_1_facebook_likes 7
gross                884
genres               0
actor_1_name         7
movie_title          0
num_voted_users      0
cast_total_facebook_likes 0
actor_3_name         23
facenumber_in_poster 13
plot_keywords        153
movie_imdb_link       0
num_user_for_reviews 21
language             14
country              5
content_rating        303
budget               492
title_year           108
actor_2_facebook_likes 13
imdb_score            0
aspect_ratio         329
movie_facebook_likes 0
dtype: int64
Movie_ID      0
Year          0
Name          0
dtype: int64

```

```

In [74]: # info about dataset after cleaning and stuff
# Dataset 1
print("INFO OF DATASET ONE")
df1.info()
print("-" * 40)

# Dataset 2
print("INFO OF DATASET TWO")
df2.info()

```

INFO OF DATASET ONE

<class 'pandas.core.frame.DataFrame'>

Index: 3722 entries, 0 to 5042

Data columns (total 28 columns):

#	Column	Non-Null Count	Dtype
0	color	3722 non-null	object
1	director_name	3722 non-null	object
2	num_critic_for_reviews	3722 non-null	float64
3	duration	3722 non-null	float64
4	director_facebook_likes	3722 non-null	float64
5	actor_3_facebook_likes	3722 non-null	float64
6	actor_2_name	3722 non-null	object
7	actor_1_facebook_likes	3722 non-null	float64
8	gross	3722 non-null	float64
9	genres	3722 non-null	object
10	actor_1_name	3722 non-null	object
11	movie_title	3722 non-null	object
12	num_voted_users	3722 non-null	int64
13	cast_total_facebook_likes	3722 non-null	int64
14	actor_3_name	3722 non-null	object
15	facenumber_in_poster	3722 non-null	float64
16	plot_keywords	3722 non-null	object
17	movie_imdb_link	3722 non-null	object
18	num_user_for_reviews	3722 non-null	float64
19	language	3722 non-null	object
20	country	3722 non-null	object
21	content_rating	3722 non-null	object
22	budget	3722 non-null	float64
23	title_year	3722 non-null	float64
24	actor_2_facebook_likes	3722 non-null	float64
25	imdb_score	3722 non-null	float64
26	aspect_ratio	3722 non-null	float64
27	movie_facebook_likes	3722 non-null	int64

dtypes: float64(13), int64(3), object(12)

memory usage: 843.3+ KB

INFO OF DATASET TWO

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 17770 entries, 0 to 17769

Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
0	Movie_ID	17770 non-null	int64
1	Year	17770 non-null	int64
2	Name	17770 non-null	object

dtypes: int64(2), object(1)

memory usage: 416.6+ KB

Use the IQR method to identify and remove outliers.

```
In [75]: df_cleaned_1=df1
# Select the column to clean
column = ['num_critic_for_reviews','gross','duration','director_facebook_likes',

# Calculate Quantiles
for col in column:
    Q1 = df_cleaned_1[col].quantile(0.25)
    Q3 = df_cleaned_1[col].quantile(0.75)
```

```

# Calculate IQR
IQR = Q3 - Q1

# Define Bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify Outliers (for reporting)
outliers = df_cleaned_1[(df_cleaned_1[col] < lower_bound) | (df_cleaned_1[col] > upper_bound)]
print(f"Number of outliers detected in {col}: {len(outliers)}")

# Remove Outliers
df_cleaned_1 = df_cleaned_1[(df_cleaned_1[col] >= lower_bound) & (df_cleaned_1[col] <= upper_bound)]

print(f"Final shape after removing outliers: {df_cleaned_1.shape}")

```

Number of outliers detected in num_critic_for_reviews: 148
 Number of outliers detected in gross: 238
 Number of outliers detected in duration: 102
 Number of outliers detected in director_facebook_likes: 350
 Number of outliers detected in num_user_for_reviews: 189
 Number of outliers detected in num_voted_users: 165
 Final shape after removing outliers: (2530, 28)

Merging the dataset

```

In [76]: # Remove Leading/trailing whitespaces and convert to lowercase
df_cleaned_1['movie_title'] = df_cleaned_1['movie_title'].str.strip().str.lower()
df2['Name'] = df2['Name'].str.strip().str.lower()

# Now perform the merge
df_merged = pd.merge(df_cleaned_1, df2, left_on='movie_title', right_on='Name',

```

```

In [77]: # Drop duplicates based on the movie title column
df_merged = df_merged.drop_duplicates(subset=['movie_title'], keep='first')

print(f"Shape after dropping duplicates: {df_merged.shape}")

```

Shape after dropping duplicates: (1238, 31)

```

In [78]: df_merged.head()

```

Out[78]:

	color	director_name	num_critic_for_reviews	duration	director_facebook_likes	actor
--	-------	---------------	------------------------	----------	-------------------------	-------

0	Color	Robert Zemeckis	240.0	96.0	0.0	
---	-------	-----------------	-------	------	-----	--

3	Color	James Bobin	218.0	113.0	33.0	
---	-------	-------------	-------	-------	------	--

4	Color	Robert Zemeckis	287.0	115.0	0.0	
---	-------	-----------------	-------	-------	-----	--

5	Color	Dean Parisot	135.0	90.0	23.0	
---	-------	--------------	-------	------	------	--

6	Color	Breck Eisner	163.0	124.0	42.0	
---	-------	--------------	-------	-------	------	--

5 rows × 31 columns



In [79]:

```
scaler_std = StandardScaler()
# Apply Standardization (Mean=0, Std=1)
df_merged = scaler_std.fit_transform(df_merged.select_dtypes(include='number'))
df_merged_normalized = pd.DataFrame(df_merged)
```

In [80]: df_merged_normalized.head()

Out[80]:

	0	1	2	3	4	5	6	7
--	---	---	---	---	---	---	---	---

0	2.674537	-0.593676	-0.786447	9.422627	1.416395	3.417107	0.855391	3.581640
---	----------	-----------	-----------	----------	----------	----------	----------	----------

1	2.276031	0.522503	-0.500765	10.416679	3.886911	1.451684	-0.538426	6.381793
---	----------	----------	-----------	-----------	----------	----------	-----------	----------

2	3.525890	0.653818	-0.786447	0.440372	1.416395	1.622935	2.741488	2.059570
---	----------	----------	-----------	----------	----------	----------	----------	----------

3	0.772577	-0.987622	-0.587336	-0.286280	-0.497468	2.530547	1.884129	-0.529922
---	----------	-----------	-----------	-----------	-----------	----------	----------	-----------

4	1.279766	1.244736	-0.422852	0.325062	0.630322	1.187361	0.987142	0.441466
---	----------	----------	-----------	----------	----------	----------	----------	----------



In []:

In []: